

```

cudaMalloc( (void**)&dev_c, N * sizeof(int) );

// copy the arrays 'a' and 'b' to the GPU
cudaMemcpy( dev_a, a, N * sizeof(int), cudaMemcpyHostToDevice );
}
cudaMemcpy( dev_b, b, N * sizeof(int), cudaMemcpyHostToDevice );

```

As in the above code, similar to the allocation in C, variables a, b and c are allocated memory on the CPU. `cudaMalloc()` is a standard sub-routine of the CUDA API to allocate memory on the device. It works similar to the C `malloc()` function we used earlier.

Since you cannot modify the memory allocated to the device from the host directly in CUDA, `cudaMemcpy()` is used to transfer data to the device. This method takes a pointer to the local memory, a pointer to the GPU memory being copied to, the number of bytes that will be copied, and a flag which determines the direction of the memory transfer, respectively.

```
vector_add<<<numBlock,numThread>>>>( dev_a, dev_b, dev_c );
```

This line is a call to the device kernel from the host to execute the function on the device. It is similar to the serial function call with some additional code. Blocks are organised in three dimensional grids and threads are organised in three dimensional blocks. `numBlock` and `numThread` are passed as arguments to let the device know about the structure to be adopted for computation.

```
cudaMemcpy( c, dev_c, N * sizeof(int), cudaMemcpyDeviceToHost );
```

This copies the resultant data back from the GPU to the host. As you can see, it is similar to the `cudaMemcpy()` we used above, with the last variable being changed to `cudaMemcpyDeviceToHost` to indicate data transfer is between device to host.

The rest of the code is pretty much self-explanatory, except that you use `cudaFree()` to free the memory allocated to the GPU.

Now that you understand the code well, compile the code to verify your results.

Save the code and `parallel_vector.cu`, and type the following command in the terminal:

```
nvcc parallel_vector.cu -o parallel
```

Now, to execute the code, run the following command:

```
./parallel
```

If you have followed this guide correctly, it will print the vectors 'a' and 'b' along with their additional resultant 'c'. This would verify that the first GPU code which you wrote has worked correctly. Still confused? Well, have a look at Figure 2, which will give you a clear understanding of how things work in parallel, in this case.

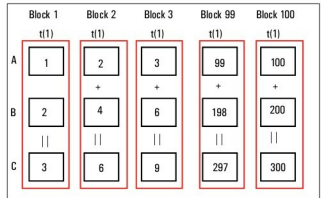


Figure 2: Parallel vector addition

Performance

You may wonder how the GPU code can perform about 100 times faster than the CPU code, since we have created 100 blocks that are executing in parallel. This is not the case, since there is an overhead involved in copying data between the CPU and the GPU and the resultant data back to the CPU. Hence, CUDA is generally used for computing algorithms that are significantly data intensive, as it would then make sense to spend some time for data transfer. GPUs are, therefore, generally known as data intensive computational devices.

As a next step, you could try programming matrix addition on the GPU in parallel to get a good grip on kernels, threads and parallel execution. Your best companion for this would be the links and the books mentioned in the 'References' section at the end of this article. I also recommend you visit the NVIDIA website and documentation, as it will give you a good idea about the power of CUDA if you are not already impressed by what this simple GPU device on your laptop can do. Next up in this series, I might cover an advanced CUDA program with multiple threads and blocks on a grid, and analyse the running time of the code. I will follow it up with a discussion on OpenACC and other simpler parallel programming models that have come up recently.

Till then, start thinking of algorithms in parallel. The world of parallel computing is here to stay! 

References

- [1] "CUDA C Programming Guide" by NVIDIA; <http://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html>
- [2] "CUDA Application Design and Development" by Rob Farber
- [3] "Programming Massively Parallel Processors" by David B. Kirk and Wen-mei W. Hwu

By: Tejaswi Agarwal

A FOSS enthusiast, the author is passionate about compute power utilisation, run time and memory utilisation of algorithms. Some of his research areas are computer architecture, parallel programming and performance engineering. He can be contacted at tejaswi.agarwal2010@vit.ac.in